

Stacks

A library lending app, from commit to cluster

DevOps Final Capstone

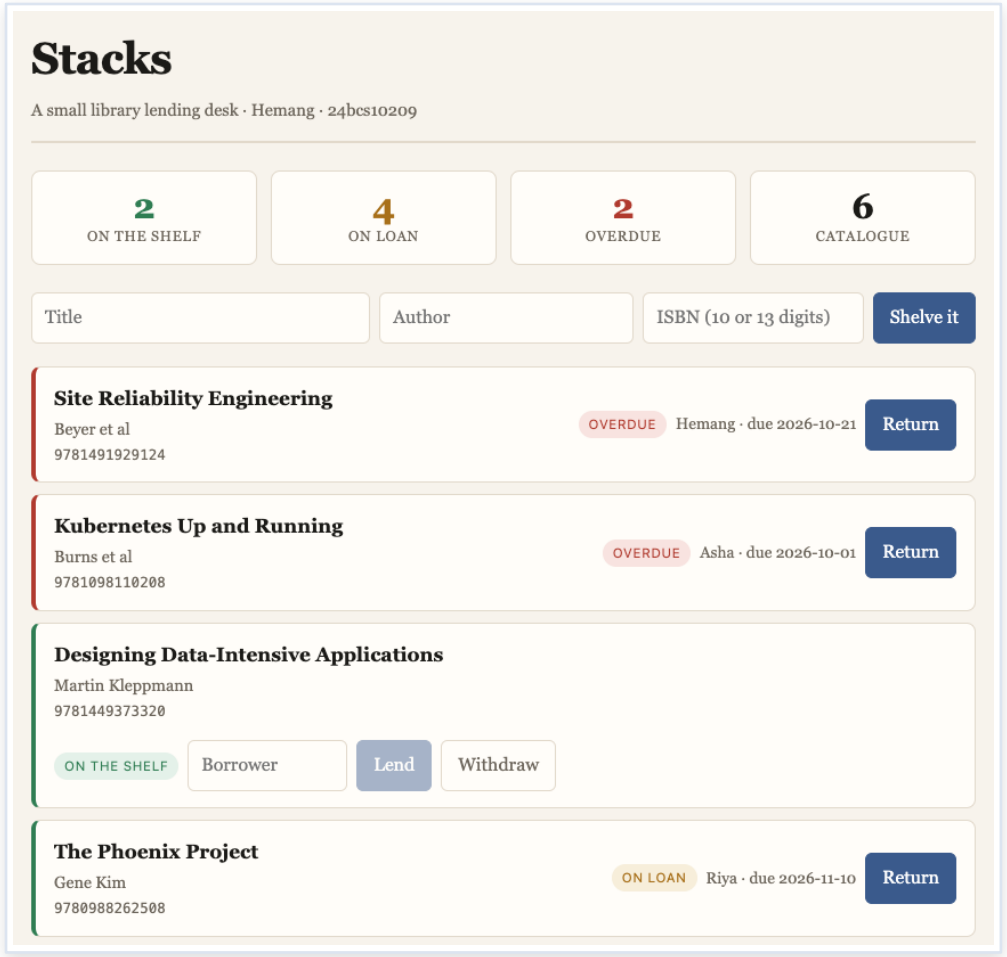
Hemang · Enrollment 24bcs10209

What I built

A catalogue of physical copies. Each copy is on the shelf or on loan, and a loan turns overdue on its own once the due date passes.

9	20	95%
API routes	pytest cases	coverage

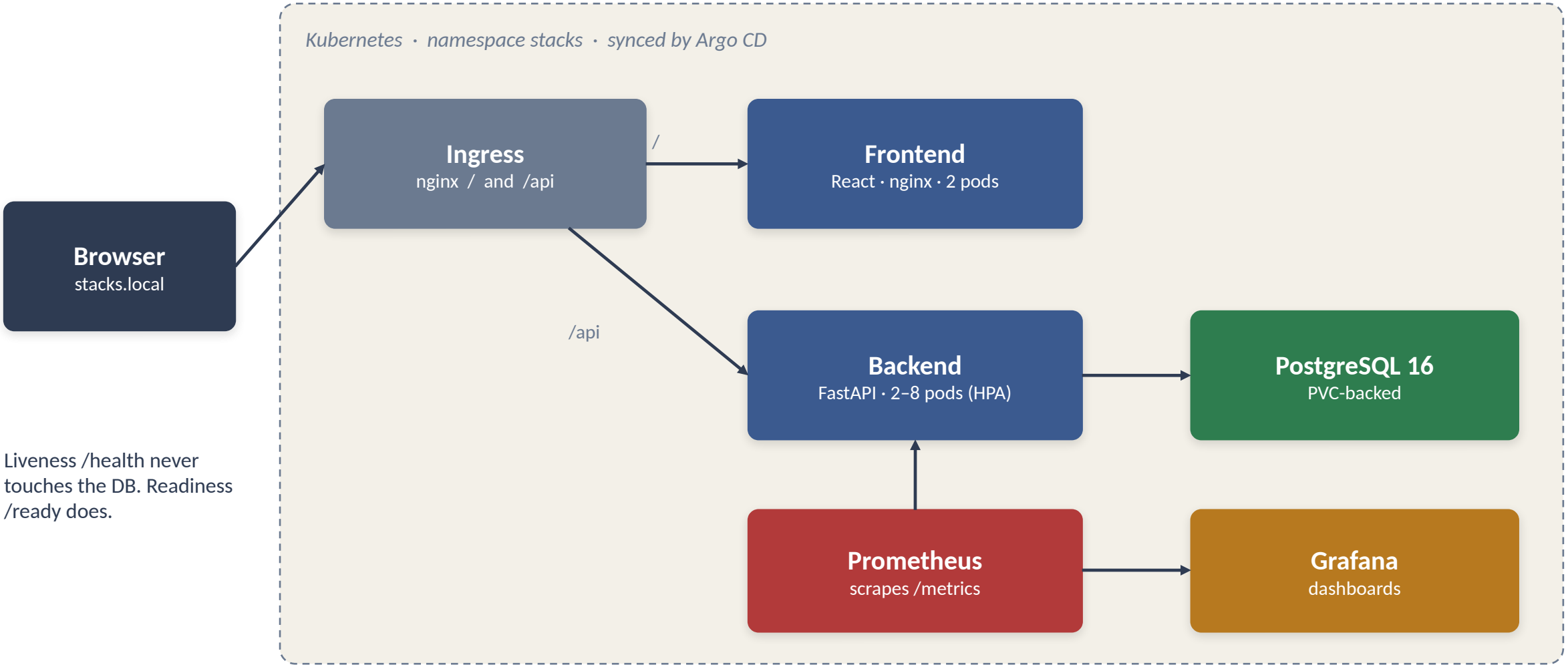
- A copy cannot be lent twice or withdrawn while out
- Loans default to 14 days, configurable without a rebuild
- "Overdue" is derived from the due date, never stored
- Duplicate ISBNs blocked by a UNIQUE index, not only code



Tech stack

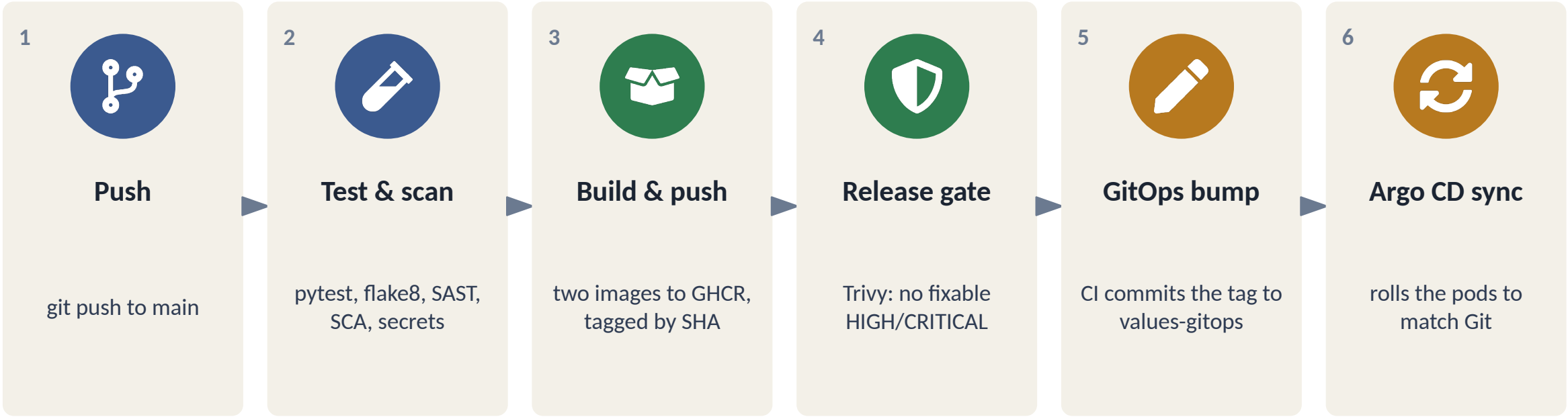
Backend	Python 3.12, FastAPI, SQLAlchemy, Alembic
Frontend	React 18 + Vite, served by nginx
Database	PostgreSQL 16, SQLite in tests
Containers	Docker multi-stage, non-root, Compose
CI/CD	GitHub Actions, GHCR, SHA image tags
Security	Trivy, bandit, pip-audit, gitleaks
Infrastructure	Terraform, AWS VPC + EKS, LocalStack
Orchestration	Kubernetes, Helm 3, Ingress, HPA
GitOps	Argo CD, auto-sync with self-heal
Observability	Prometheus, Grafana

Architecture



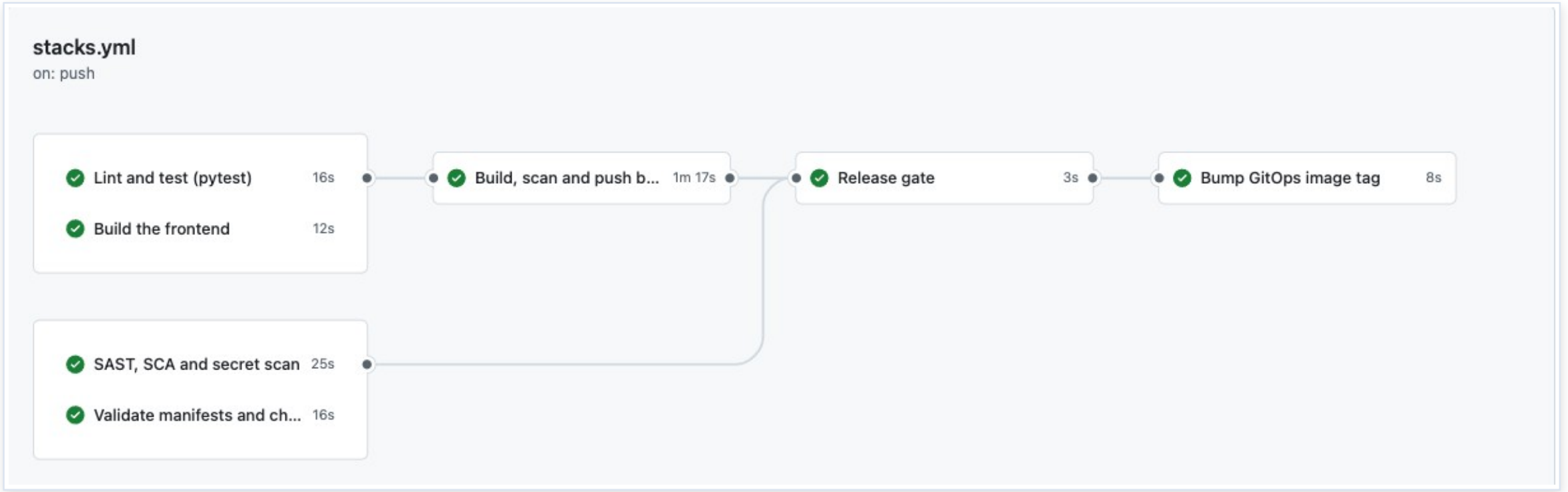
Liveness /health never touches the DB. Readiness /ready does.

From commit to cluster



CI never runs kubectl. It writes one file in Git; Argo CD does the deploying. A rollback is a git revert.

CI/CD: seven gated jobs



7 jobs

test, frontend, security and manifests run in parallel

SHA tags

image tag = 12-char commit SHA, never latest

1m 55s

push to deployable images, end to end

DevSecOps: four scanners, one gate

SAST

bandit

our own source for insecure patterns

SCA

pip-audit

third-party dependencies with known CVEs

Secrets

gitleaks

full Git history for committed credentials

Images

Trivy

both images, OS and library packages

The gate fails only on fixable HIGH or CRITICAL findings.
An unfixable CVE that blocks every build becomes a gate people route around. Both images also run as non-root users, set in the Dockerfile.

Infrastructure as code: Terraform

```
########## PLAN ##########
$ terraform plan -no-color -out=tfplan 2>&1 | grep -E '^  # |^Plan:'
  # aws_eks_cluster.main will be created
  # aws_eks_node_group.main will be created
  # aws_iam_role.cluster will be created
  # aws_iam_role.node will be created
  # aws_iam_role_policy_attachment.cluster_policy will be created
  # aws_iam_role_policy_attachment.node_cni will be created
  # aws_iam_role_policy_attachment.node_registry will be created
  # aws_iam_role_policy_attachment.node_worker will be created
  # aws_internet_gateway.main will be created
  # aws_route_table.public will be created
  # aws_route_table_association.public[0] will be created
  # aws_route_table_association.public[1] will be created
  # aws_subnet.public[0] will be created
  # aws_subnet.public[1] will be created
  # aws_vpc.main will be created
Plan: 15 to add, 0 to change, 0 to destroy.
```

terraform plan: 15 resources, VPC, subnets, routing, IAM and EKS

Remote-state ready, terraform.tfvars.example only, and **no credentials in Git**

13 / 15

resources applied: VPC 10.30.0.0/16, two public subnets in two AZs, internet gateway, routes and IAM roles



Stated plainly: no EKS cluster

There is no AWS account behind this project. LocalStack's free tier returns HTTP 501 for CreateCluster, so the two EKS resources do not apply. minikube stands in for the cluster.

Kubernetes and Helm

One chart, three value sets

values.yaml, values-dev.yaml, values-prod.yaml; Postgres migrates in an init container

HPA: 2 to 8 pods

backend scales on 60% CPU, metrics-server supplies the samples

Ingress splits one host

/ goes to the React SPA, /api to the backend, no CORS

Data survives pods

Postgres on a PVC; deleting its pod keeps every loan

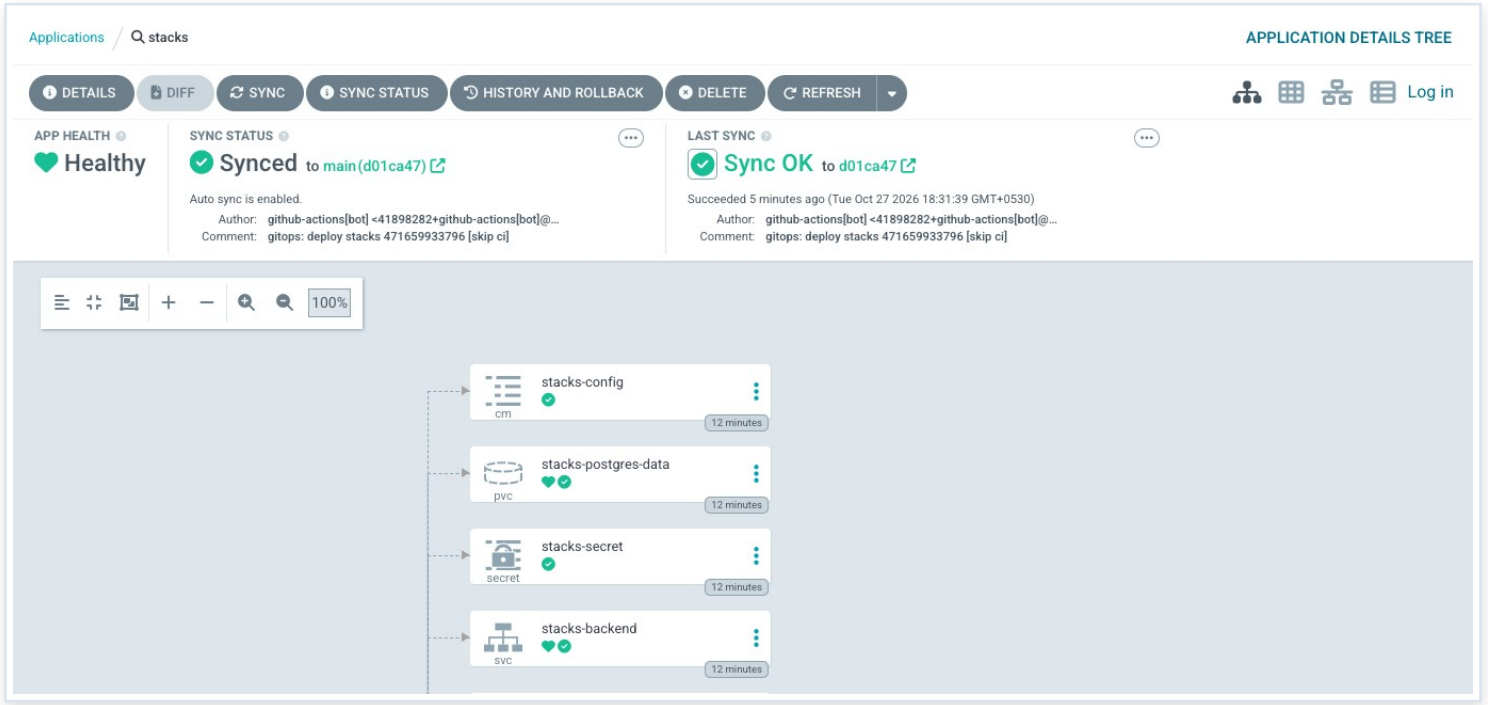
```
########## INGRESS SPLITS / AND /api ##########
$ kubectl -n stacks describe ingress stacks | sed -n '/Rules:/,/Annotations:/p'
Rules:
  Host          Path  Backends
  ----          ----  --------
  stacks.local
                /(api/.*)   stacks-backend:8000 (10.244.0.59:8000,10.244.0.61:8000)
                /(.*)       stacks-frontend:3000 (10.244.0.58:3000,10.244.0.60:3000)
Annotations:    meta.helm.sh/release-name: stacks

$ minikube ssh -- "curl -s -o /dev/null -w 'GET  /          -> HTTP %{http_code}  (frontend)\n' -H 'Host: stacks
  .local' http://192.168.49.2/"
GET  /          -> HTTP 200  (frontend)
```

```
########## HPA ##########
# metrics-server needs a minute to produce the first samples
$ kubectl -n stacks get hpa
NAME            REFERENCE                  TARGETS        MINPODS   MAXPODS   REPLICAS   AGE
stacks-backend  Deployment/stacks-backend  cpu: 10%/60%   2         8         2          2m22s

$ kubectl -n stacks describe hpa stacks-backend | sed -n '/Metrics:/,/Events:/p' | head -8
Metrics:                                               ( current / target )
  resource cpu on pods  (as a percentage of request):  10% (5m) / 60%
Min replicas:                                          2
Max replicas:                                          8
Deployment pods:                                       2 current / 2 desired
Conditions:
```

GitOps with Argo CD



Synced to the bot's commit d01ca47: "gitops: deploy stacks 471659933796"

Git is the source of truth
the Application renders helm/stacks from main

prune: true
deleted from Git, deleted from the cluster

selfHeal: true
a manual kubectl edit is reverted

[skip ci] on the bump
it moves the cluster, not the pipeline

Observability: Prometheus and Grafana



Live dashboard during the Argo CD rollout: old pods hand over to new ones

- `stacks_http_requests_total`
- `stacks_http_errors_total (5xx, probes excluded)`
- `stacks_not_ready_total`
- `stacks_books_total`, `stacks_books_overdue`

Prometheus finds the pods itself. Kubernetes service discovery plus pod annotations means a new replica is scraped without editing any config. Both backend pods show as UP.

Key learnings

1 **Derive what time decides**

Storing OVERDUE would be wrong every midnight. Computing it from the due date cannot drift.

2 **Put the constraint in the database**

The handler's ISBN check is a nice message; the UNIQUE index is what prevents duplicates.

3 **Route order is behaviour**

/api/books/stats declared after /{id} parses as an id and returns 422. It needs a test.

4 **A readiness 503 is not an error**

Counting it as one turns every healthy rollout into an alert.

A commit is the only input

Tests, scans, the registry and the cluster all follow from it.

github.com/hemangtk/DevOps
Final DevOps Project/ · Hemang · 24bcs10209